Contents

1. Executive summary 1

2. Sources and integration map 1

3. System architecture 2

4. Fidelity evidence 3

5. Interface redesign 4

6. Verification and test results 5

7. Measured performance 6

8. Limitations and remaining requirements 6

1. Executive summary

This report documents the transformation of two research artifacts - a static Three.js demonstration that replayed pre-recorded simulations, and the EPtask V7.1 executed notebook that produced those simulations - into a single live application. The delivered system is an operations console: a Next.js 16 frontend over a Python FastAPI service that executes the notebook's original modules on every request. Nothing visible on the simulation pages is pre-computed; each launched run is computed at request time, and the repository's tests prove that the live outputs are identical to the notebook's recorded ones.

The integration preserves the demo's strongest asset - its 3D situation map of the Trishuli valley corridor - while removing its embedded 7.2 MB dataset, decorative bloom effects, and scripted story machinery. The notebook's strongest assets - the corridor co-simulation, the vehicular edge-computing (VEC) cell environment, the trained MEPPO policies, and the paired outage protocol - now serve live requests through their original, unmodified code. Where the original sources were assumptions or static artifacts, the new application keeps them static and says so in the interface.

All acceptance criteria from the integration brief were verified: the main workflow operates end to end in a real browser; notebook-derived outputs match the original implementation exactly; errors and missing dependencies are handled with actionable messages; and setup instructions allow another developer to reproduce the verified environment. The measured performance figures and the remaining limitations are reported honestly in sections 7 and 8.

2. Sources and integration map

Both archives were inventoried before any change. The demo is a self-contained HTML/Three.js application whose data pipeline is documented by its own build tools: tools/make_traces.py converts a notebook results bundle into the replayable demo_data.json, and tools/build_demo.py inlines that JSON into the page. The notebook archive contains the v7.1 full run (16 Python modules, checkpoints, ONNX exports, results bundle) plus earlier v5.1/v6.1 notebooks and v7.2 real-data attempts.

Demo element	Old demo	New application
3D situation map (terrain, routes, vehicles, RSUs, playback)	embedded replay	ported renderer, driven by live API traces; restrained operational restyle
Corridor traces (12 presets)	embedded demo_data.json	corridor.run_trial executed per request; recorder verified against an unrecorded rerun
Convoy behaviour	embedded	live; one trial per member with the same blockage, traffic seed and RSUs
Cell tick replay (520 ticks)	embedded	VECEnv + OnnxPolicy live, with env.peek counterfactuals
C3 outage table	static table	paired live study via policies.rollout
results_v71.json numbers and figures	mixed into panels	served as explicitly labeled reference artifacts

Table 1. Where each demo element comes from now.

The v7.1 line was selected as the integration source because the demo replayed exactly that run (20261008-124455-ba2667). The v7.2 real-data attempts could not be integrated: they require live internet access to OpenStreetMap and NASA COOLR, which the original environment itself could not reach - the notebook rejected its own fetched OSM extract for disconnected roads and documented the COOLR server returning 404. This is documented rather than papered over; the application keeps the notebook's schematic two-route graph through the real place positions.

3. System architecture

The application separates presentation, orchestration and simulation. The Next.js console owns forms, tables, charts and the 3D map; the FastAPI service owns the notebook world - it imports the original modules from its core/ directory, loads the notebook's own bundle assets once at startup, and exposes a small, validated API. In the sandbox the two are joined by the platform gateway; in a plain deployment the frontend proxies to the service port directly. No database, queue or cloud service was added because no requirement justifies one.

Layer	Component	Responsibility
Frontend	Next.js 16 console (single route)	launchers, playback, metrics, decision inspector, reference browser
Gateway	Caddy reverse proxy	single external port; routes /api to the service by query parameter
Service	FastAPI (port 8100)	validation, job queue, JSON serialisation, error mapping
Simulation	app/corridor_svc.py	recorded corridor.run_trial runs + hook verification
Simulation	app/cell_svc.py	VECEnv runs driven by shipped ONNX actors with explanations
Simulation	app/outage_svc.py	paired outage studies via policies.rollout and runlog.mean_ci
Assets	core/ + assets/demo_bundle/	the unmodified notebook modules and its recorded bundle

Table 2. Service boundaries.

Requests that take longer than a moment are executed as jobs on a two-worker thread pool. The client submits, receives a job id, and polls; progress is reported by the simulation itself - per convoy member, per tick block, or per rollout - never fabricated. Job results are kept in a bounded in-memory registry (40 most recent) so the session's run history survives page navigation without introducing persistence the task does not need.

Every input is validated server-side with strict ranges: vehicle counts to 60, ticks to 3000, at most six paired seeds and four outage sizes per study, footprints between 500 and 30,000 m. No user input ever reaches a filesystem path; reference sections and figures are allow-listed. The service returns structured errors with actionable text and never exposes stack traces to the client.

4. Fidelity evidence

The notebook's simulations are deterministic given a seed and parameters, which makes equivalence testable rather than aspirational. The original demo embedded traces produced by tools/make_traces.py from the notebook run; the live service re-executes the same code with the same inputs and the test suite compares the results field by field.

Consistency check	Scope	Result
Corridor ambulance tracks	12 preset traces (3 hazards x 4 warning modes)	identical, all rows
Corridor civilian tracks	same 12 traces	identical, all rows
Corridor summaries	arrival/oracle/extra/informed, flags	equal within 1e-9
Cell trace	520 ticks, model B v30, V=30, seed 21	identical, field-for-field
Settled true times	510 settled requests	equal within 5e-4 s
Determinism on repeat	flood/RSU run twice, landslide repeat over HTTP	identical summaries
Recorder-hook neutrality	every recorded run vs unrecorded rerun	verified at request time; service refuses on mismatch

Table 3. Output consistency against the notebook recording (scripts/test_consistency.py and tests/test_service.py).

Two fidelity mechanisms deserve mention. First, the recorder is the demo builder's own anchored-source-patch of corridor.run_trial; the service re-derives the patched function at startup, aborts if the anchor lines ever move, and re-runs each recorded trial unrecorded to prove the hook changes nothing before serving a trace. Second, policy decisions come from the shipped ONNX exports through the notebook's own OnnxPolicy wrapper; the notebook itself measured zero decision disagreements out of 500 between the exports and the torch checkpoints, so evaluating the exports is a documented, verified equivalence rather than a shortcut.

5. Interface redesign

The demo's holographic aesthetic - bloom, aurora terrain, stars, neon grid, scripted damage animations and an eight-step story - was replaced by a restrained operational design: a graphite terrain with subtle contours and elevation shading, desaturated road colours, status colour reserved for meaning (green alive, red down, amber attention), tabular numerals, and clear empty and loading states. Animation survives only as meaningful feedback: playback of the returned trace, an ambulance trail showing the travelled route, and progress bars whose stages name the real simulation work in flight.

The console is organised into six sections. Overview carries provenance, service health and the model registry. Corridor launches road-scale trials and plays the returned trace on the map with scrubber and speed controls, alongside metric cards, the convoy table and the coverage curve. VEC cell renders the 558-metre highway with its eight roadside units and two traffic lanes on a canvas strip, a per-tick timeline of completion times against deadlines, and a decision inspector showing the policy's probabilities and exact counterfactuals for the selected request. Outage study runs the paired protocol and tabulates mean plus 95% confidence intervals with the notebook's recorded table available for context. Reference browses the recorded artifacts and figures. About documents the integration inside the product itself.

Accessibility and responsiveness were addressed directly: semantic landmarks and aria labels on charts and controls, keyboard operation of the map (arrow keys seek), 44 px touch targets, and layouts that collapse from a seven-column launcher grid to stacked mobile forms. Without WebGL the map degrades to a notice

while every table and chart remains functional; without the Python service the header reports it and each panel explains what is missing.

6. Verification and test results

The backend suite (pytest, 40 tests) covers asset integrity, startup and health, unit behaviour of each service, validation and invalid-request handling, output consistency against the notebook recording, missing-file failure modes, repeated-request determinism and the end-to-end job workflow over HTTP. All 40 pass. The frontend was verified in a real browser: every section was opened, a corridor simulation, a cell run and a paired outage study were launched through the interface, results rendered from live API data, and desktop and mobile layouts were exercised. Issues found during that verification (a label getter bug that disabled the 3D map, a stale variable in the reference panel) were fixed and re-verified.

Area	Checks	Outcome
Asset and dependency integrity	core modules present; bundle files present; imports succeed	pass
Backend startup and health	/health readiness; /api/meta provenance; /api/graph geometry	pass
Unit behaviour	corridor service, cell service, outage service incl. baselines	pass
Validation and invalid requests	422 with field-level messages; unknown job/section 404	pass
Output consistency vs notebook	12 corridor traces + 520-tick cell trace + summaries	pass (exact)
Failure modes	missing model export raises actionable FileNotFoundError	pass
Repeated requests	deterministic repeat; job listing; LRU bound	pass
End-to-end over HTTP	submit, poll, complete corridor job; results sane	pass
Frontend (browser)	all six sections; three live runs; desktop + mobile viewports	pass after fixes

Table 4. Verification summary (mini-services/eptask-api/tests).

To reproduce: start the service (bash mini-services/eptask-api/run.sh), install its requirements, then run python3 -m pytest tests/ -v from mini-services/eptask-api. The consistency tests locate the original demo recording from the extracted demo.zip and can be pointed at any copy through the EPTASK_DEMO_DATA environment variable.

7. Measured performance

Timings below were measured on the deployed sandbox (CPU-only, torch 2.14.1+cpu, the same build the notebook used). Startup loads the graph, reference bundle and latency arms in about two seconds; ONNX sessions load in about 20 ms each and are cached after first use, so repeated requests pay only simulation cost.

Operation	Measured	Notes
Corridor run with convoy and trace	8-14 s wall	7 full 6-hour trials plus one unrecorded verification rerun per request
Corridor run without convoy	1-2 s	ambulance trial + verification
VEC cell, 520 ticks, explanations on	0.6-2 s	dominated by env.peek counterfactuals, as in the notebook
Paired outage study, 2 policies x 3 sizes x 3 seeds	6-10 s	36 rollouts of 1101 steps; linear in policies x sizes x seeds
Concurrent behaviour	2 workers	further jobs queue with honest 'queued' progress
Frontend first render	under 2 s	trace payloads for one corridor run are 0.3-1 MB

Table 5. Measured timings (sandbox, CPU).

No algorithm, parameter or preprocessing change was made for speed; the numbers above are the original notebook code's cost of doing real work per request. The dominant optimisations are architectural: models and assets load once, sessions are cached, and heavy runs are bounded by the worker pool instead of blocking the HTTP event loop.

8. Limitations and remaining requirements

The corridor graph is the notebook's schematic two-route map through real place positions; live OpenStreetMap ingestion is not implemented, and the notebook's own v7.2 attempt documents why its fetched extract was rejected. Outage and cell studies evaluate the ONNX exports rather than the torch checkpoints - a verified equivalence (0/500 disagreements) rather than an identity. The micro-latency samples feeding corridor service delays are the recorded arms from the bundle, selectable per request; recomputing them live would double run times for identical numbers. Runs are kept in memory only; there is no authentication layer, which is proportionate to a research console but must be added before any exposure beyond a trusted network.

Remaining setup for another developer is short: Python 3.12 with the requirements in mini-services/eptask-api/requirements.txt, Bun or Node 20 for the console, one environment variable for the service port if 8100 is unsuitable, and the two commands in the repository README. Browser verification covered headless Chromium (WebGL2) and standard desktop Chrome paths; Safari and Firefox were not exercised. Results inherit the sources' declared assumptions - schematic geometry, Poisson traffic, fixed radio ranges and the convoy's dispatch-time convention - and therefore support engineering comparison, not operational claims.